AWS Architectural Lab: Serverless REST API with API Gateway, Lambda & DynamoDB

Estimated Duration: 1 to 1.5 Hours
Focus Areas: Serverless REST API, Synchronous Request Routing, API Gateway Lambda Proxy Integration, NoSQL Data Persistence, CORS & Least Privilege IAM

Architecture Overview & Blueprint

This lab expands your AWS Solutions Architect portfolio into **Synchronous Serverless RESTful APIs**. Building upon asynchronous event buffering (SQS), this architecture focuses on real-time, synchronous HTTP request handling using Amazon API Gateway, AWS Lambda, and Amazon DynamoDB without running traditional web servers.

Component	AWS Service	Architectural Purpose
API Entrypoint	Amazon API Gateway	Exposes RESTful endpoints (/items), handles SSL, CORS, and routes HTTP requests (GET, POST) synchronously.
Compute Layer	AWS Lambda	Executes business logic statelessly in Python to parse incoming request payloads and execute database operations.
Database Tier	Amazon DynamoDB	Stores product records as JSON document items using low-latency NoSQL storage.

```
[ HTTP Client / Browser ] ⟶ (HTTP GET/POST) ⟶ [ Amazon API Gateway ] ⟶ (Lambda Proxy) ⟶ [ AWS Lambda ] ⟶ [ DynamoDB Table ]
```

Description of the Project

In this lab, you will build a scalable serverless backend service. A client application makes synchronous HTTP REST requests (e.g., POST /items to create a product, or GET /items to fetch all products) to an public API Gateway endpoint. API Gateway passes the request context directly to an AWS Lambda function via **Lambda Proxy Integration**. The Lambda function evaluates the HTTP verb, interacts with an Amazon DynamoDB table using boto3, and formats the result into a structured JSON HTTP response back to the client.
This design highlights core SAA-C03 architectural principles: building **scalable microservices**, leveraging **Lambda Proxy Integration** for seamless request/response mapping, managing **CORS (Cross-Origin Resource Sharing)** for web clients, and enforcing **least-privilege**

database security.

Step-by-Step Implementation Guide

Step 1: Building the Database Tier

- **Create DynamoDB Table:**
  - Table Name: Products
  - Partition Key (PK): id (Type: String)
  - Capacity Mode: On-Demand (pay-per-request pricing model).

Step 2: Security & IAM Execution Role

- Create an IAM Role for Lambda named APIGatewayLambdaRole.
- Attach AWS Managed Policy: AWSLambdaBasicExecutionRole (enables writing execution logs to CloudWatch).
- Attach Inline Policy named DynamoDBCRUDPolicy for restricted database access:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "dynamodb:PutItem",
        "dynamodb:Scan",
        "dynamodb:GetItem"
      ],
      "Resource": "arn:aws:dynamodb:*:*:table/Products"
    }
  ]
}
```

Step 3: Deploying the Serverless Backend (AWS Lambda)

- Create a Python 3.11/3.12 Lambda Function named ProductsApiHandler.
- Attach the IAM execution role APIGatewayLambdaRole.
- Deploy the function code to handle both POST (create) and GET (read) requests:

```
import json
import boto3

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('Products')

def lambda_handler(event, context):
    http_method = event.get('httpMethod')

    # POST /items - Create product entry
```

```
    if http_method == 'POST':
        body = json.loads(event.get('body', '{}'))
        table.put_item(Item=body)
        return {
            'statusCode': 201,
            'headers': {
                'Content-Type': 'application/json',
                'Access-Control-Allow-Origin': '*'
            },
            'body': json.dumps({'message': 'Item created
successfully', 'item': body})
        }

    # GET /items - Retrieve all items from DynamoDB
    elif http_method == 'GET':
        response = table.scan()
        items = response.get('Items', [])
        return {
            'statusCode': 200,
            'headers': {
                'Content-Type': 'application/json',
                'Access-Control-Allow-Origin': '*'
            },
            'body': json.dumps(items)
        }

    return {
        'statusCode': 400,
        'headers': {
            'Content-Type': 'application/json',
            'Access-Control-Allow-Origin': '*'
        },
        'body': json.dumps({'error': 'Unsupported HTTP method'})
    }
```

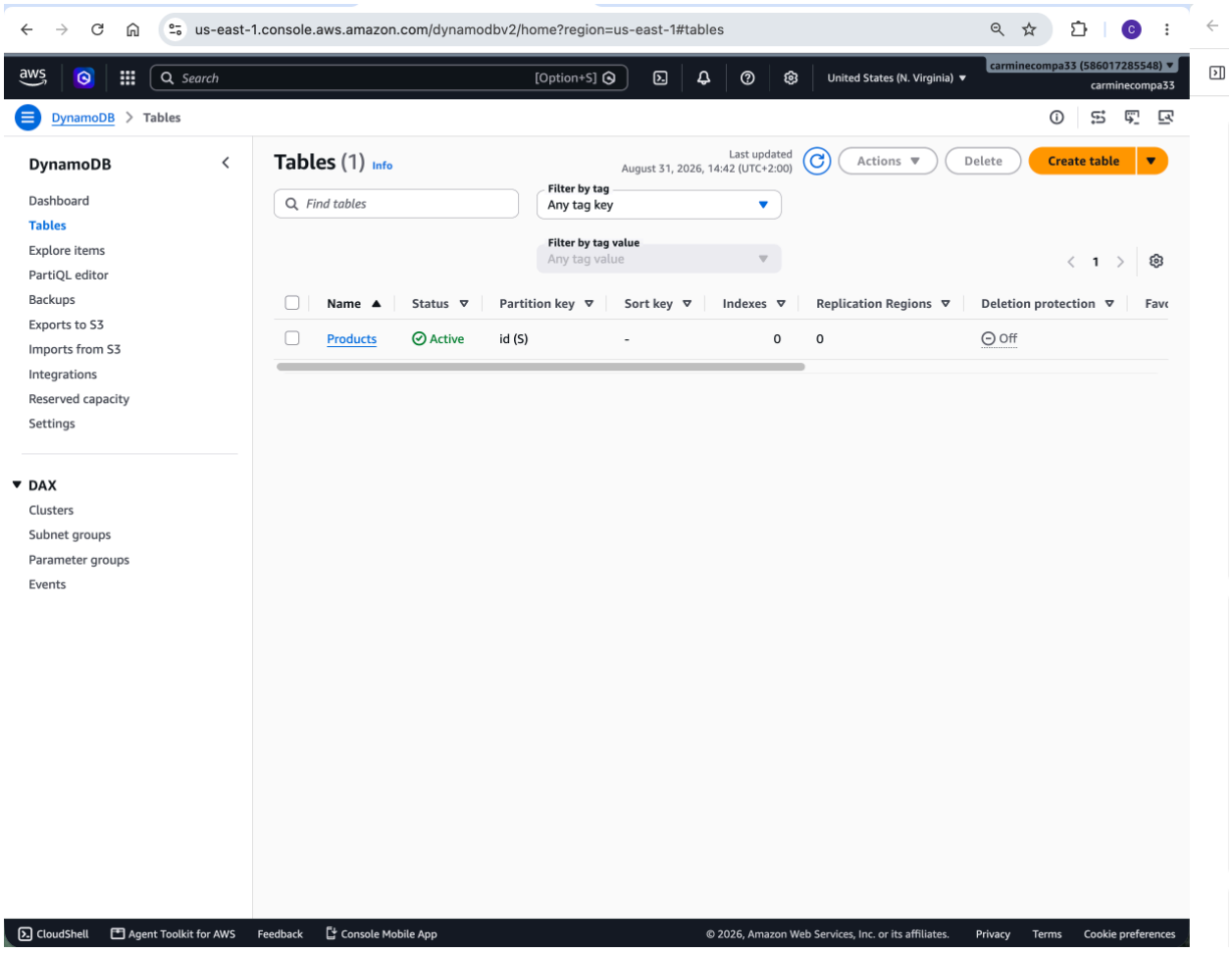
Step 4: Building & Deploying the REST API (Amazon API Gateway)

- **Create REST API:** Go to API Gateway ➔ Create API ➔ REST API (Build). Name: ProductsRESTAPI.
- **Create Resource:** Create a resource named items (Resource Path: /items). Enable CORS.
- **Create Methods:**
    - Create **POST** method on /items: Integration type ➔ Lambda Function. Enable **Use Lambda Proxy Integration**. Target function ➔ ProductsApiHandler.
    - Create **GET** method on /items: Integration type ➔ Lambda Function. Enable **Use Lambda Proxy Integration**. Target function ➔ ProductsApiHandler.
- **Deploy API:** Select Actions ➔ Deploy API. Deployment stage name: dev. Copy the generated **Invoke URL**.

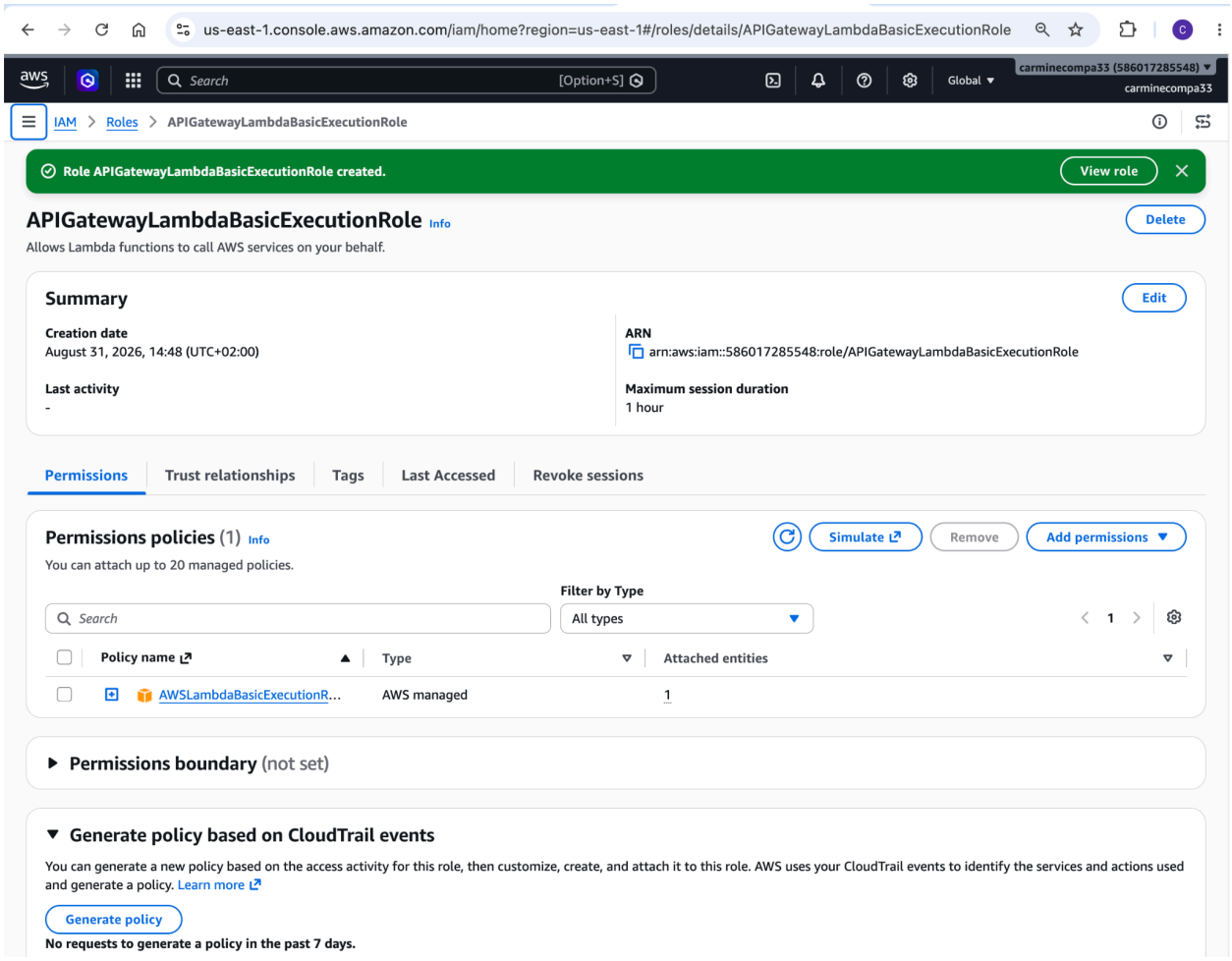
Step 5: Testing & Verification

- **Test POST Request (Create Record):** Execute a curl request (or via Postman):
  ```
  curl -X POST https://<INVOKE-URL>/dev/items \
       -H "Content-Type: application/json" \
       -d '{"id": "p101", "name": "Wireless Headphones", "price": 99}'
  ```
- **Test GET Request (Retrieve Records):** Execute a curl request to list products:
  ```
  curl https://<INVOKE-URL>/dev/items
  ```
- **Verify Database & CloudWatch:** Inspect the Products table in DynamoDB console to verify item persistence, and review CloudWatch logs under /aws/lamb
- da/ProductsApiHandler.

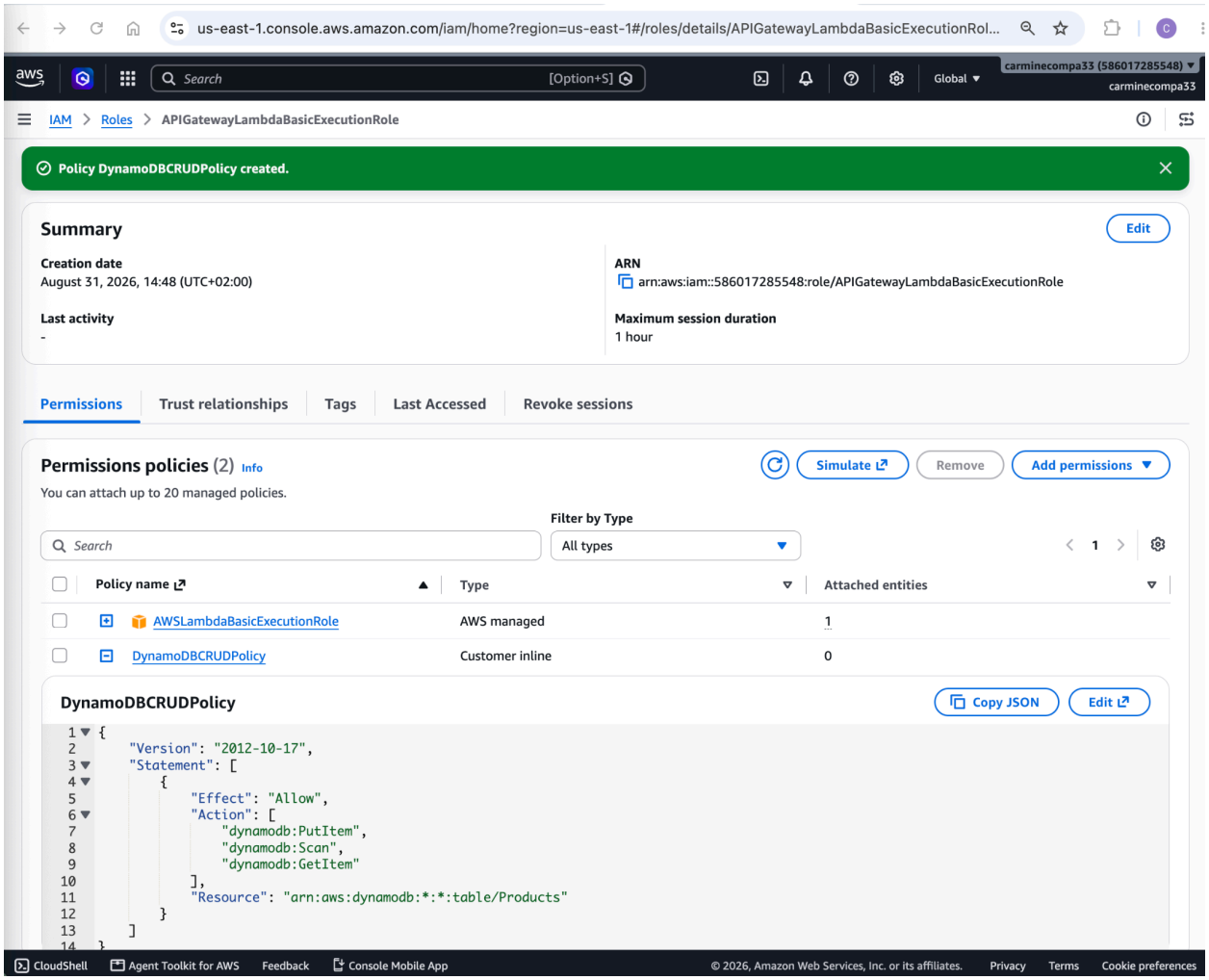
Create DynamoDB



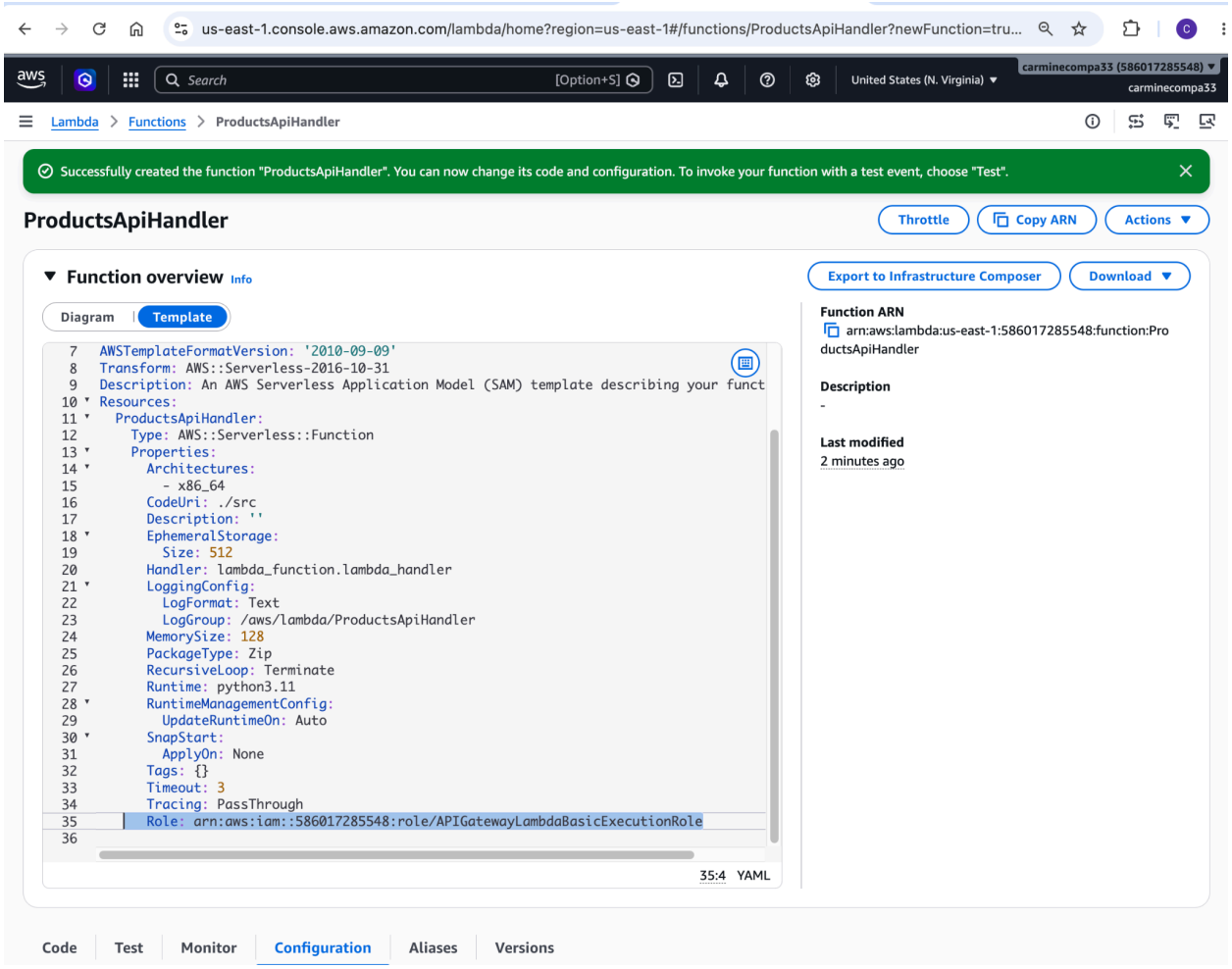
Create IAM Role with AWS LambdaBasicExecutionRole policy

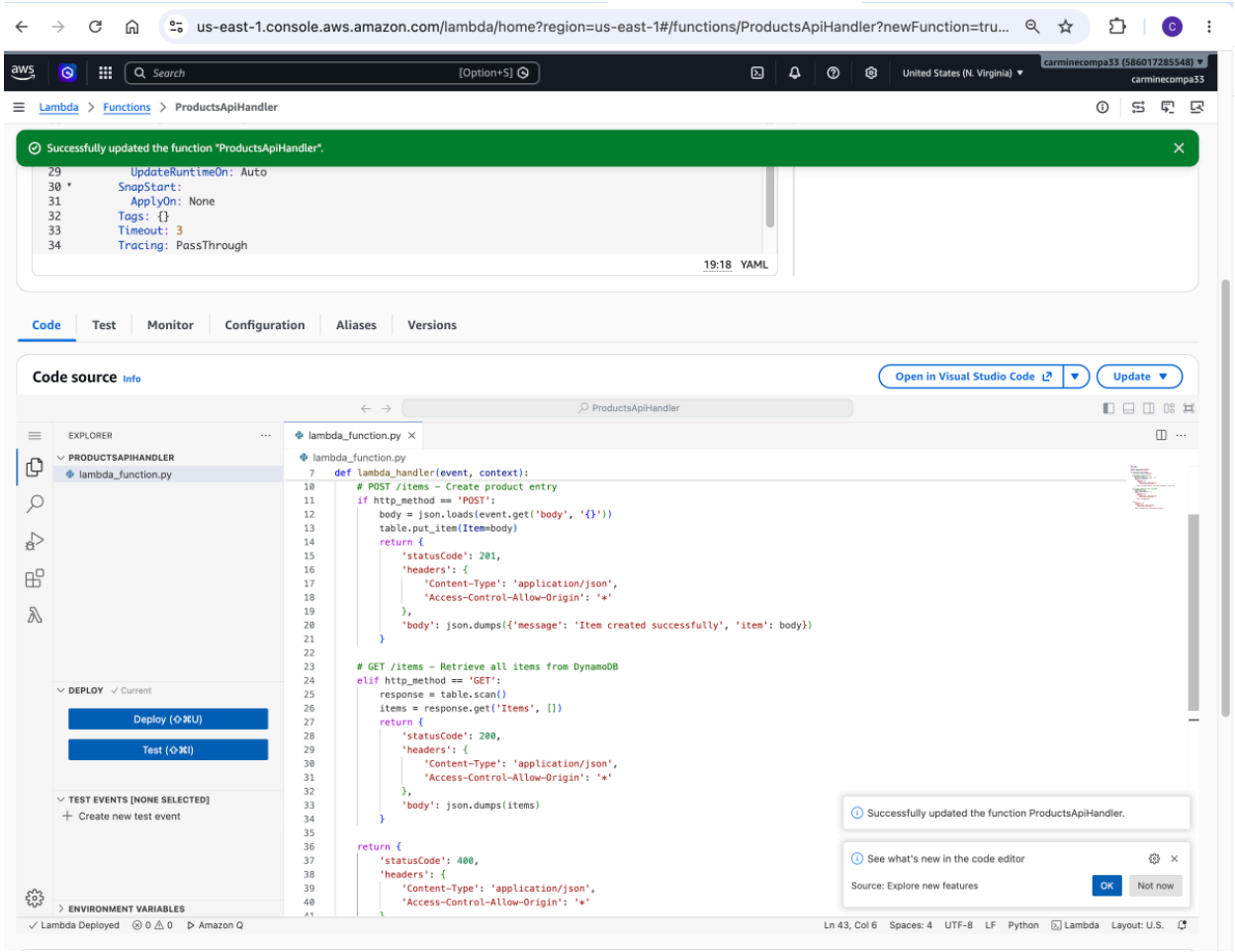


Created policy for DynamoDB CRUD operations



Created Lambda function with APIGatewayLambdaBasicExecutionRole




us-east-1.console.aws.amazon.com/lambda/home?region=us-east-1#/functions/ProductsApiHandler?newFunction=tru...
Search
[Option+S]
United States (N. Virginia)
carminecompa33 (586017285548)
carminecompa33
Lambda > Functions > ProductsApiHandler
Successfully updated the function "ProductsApiHandler".
UpdateRuntimeOn: Auto
SnapStart:
ApplyOn: None
Tags: {}
Timeout: 3
Tracing: PassThrough
19:18 YAML
Code
Test
Monitor
Configuration
Aliases
Versions
Code source Info
Open in Visual Studio Code
Update
EXPLORER
PRODUCTSAPIHANDLER
lambda_function.py
DEPLOY ✓ Current
Deploy (⇧⌘U)
Test (⇧⌘I)
TEST EVENTS [NONE SELECTED]
Create new test event
ENVIRONMENT VARIABLES
lambda_function.py
def lambda_handler(event, context):
POST /items - Create product entry
if http_method == 'POST':
body = json.loads(event.get('body', '{}'))
table.put_item(Item=body)
return {
'statusCode': 201,
'headers': {
'Content-Type': 'application/json',
'Access-Control-Allow-Origin': '*'
},
'body': json.dumps({'message': 'Item created successfully', 'item': body})
}
GET /items - Retrieve all items from DynamoDB
elif http_method == 'GET':
response = table.scan()
items = response.get('Items', [])
return {
'statusCode': 200,
'headers': {
'Content-Type': 'application/json',
'Access-Control-Allow-Origin': '*'
},
'body': json.dumps(items)
}
return {
'statusCode': 400,
'headers': {
'Content-Type': 'application/json',
'Access-Control-Allow-Origin': '*'
}
Successfully updated the function ProductsApiHandler.
See what's new in the code editor
Source: Explore new features
OK
Not now
Lambda Deployed
Ln 43, Col 6
Spaces: 4
UTF-8
LF
Python
Lambda
Layout: U.S.